Build Plan: mini-coding-agent-js

> **Goal:** Build a fully capable coding-agent harness **starting from a simple chatbot** and adding one architectural layer at a time.
> **Method:** Every stage produces a runnable program. You chat with the model at Stage 3, give it read-only tools at Stage 7, let it edit files at Stage 8, execute commands at Stage 9, remember things at Stage 10, observe its lifecycle at Stage 11, and finally wrap it in the ReAct loop at Stage 12.
> **Why:** If you build the plumbing first you wait hours before the thing talks. If you build the conversation first, every new feature is an immediate super-power you can feel.

CONTENTS

What is in this manual

Prerequisites 3

Full Architecture (end state) 4

How to Use This Plan 5

Phase A: A Chatbot That Talks 6

Phase B: Give the Chatbot Hands (Tools) 10

Phase C: Memory & Observability 13

Phase D: The Agent Loop & Polish 16

Phase E: Hardening & Docs 19

Checklist 21

Page numbers refer to the folio at the foot of each page.

Prerequisites

- Node.js >= 20 (ES modules, `AbortSignal.any`, `node:test`)
- One runtime dependency: `chalk` (v5+, ESM)
- OpenRouter API key (only needed when you want live mode; demo is offline)

Full Architecture (end state)

```
CLI (cli.js)
  ├─ loadConfig()              config.js
  ├─ Workspace + Sandbox       workspace.js / sandbox.js
  ├─ Memory                    memory.js
  ├─ chatFn (real | demo)      llm.js / demo.js
  ├─ Hooks (trace by default)  hooks.js
  └─ runTask() per task        agent.js
       └─ think → model call → tool calls? ──no──→ answer, stop
                    ↑                │
                    └──── yes: run each tool, append result
```

The golden rule (enforced from Stage 12 onward): `agent.js` is a library. It knows nothing about flags, colours, `.env`, or `process.exit`. The CLI owns all process concerns. Tests can run the loop with a fake model and no network.

Before Stage 12: `cli.js` owns the conversation loop directly. We extract it into `agent.js` only once the loop is complex enough to deserve its own file.

How to Use This Plan

1. Pick the next stage.
2. Read the **What you will learn**, **Files to create**, and **Files to modify** sections.
3. Build/modify.
4. Run the **Acceptance criteria**.
5. When satisfied, move on.

Do not skip ahead. Each stage adds a concrete capability. The CLI should do something new and useful after every stage.

Phase A: A Chatbot That Talks

The goal of this phase is a working OpenRouter chat client. No tools, no sandbox, no memory—just you and the model.

Stage 0: Scaffold

What you will learn: ES modules project layout, the single-dependency constraint, workspace directory design.

Key concepts: - All files the agent creates or edits are placed in the `workspace/` directory (the agent's working folder). This is a deliberate containment rule — the agent operates from the repo root but only writes inside `workspace/`. - The agent does not create files in subfolders of `workspace/` unless explicitly told to. Relative paths inside `workspace/` are resolved from that folder.

Files to create: - `package.json` - `.gitignore` - `workspace/.gitkeep` - `src/.gitkeep`

Acceptance criteria: - `cd mini-coding-agent-js && npm install` installs exactly `chalk`. - `node --test` reports `0 tests` (no failure). - `workspace/` exists and is git-ignored except `.gitkeep`. - `type: "module"` is set in `package.json`.

Stage 1: Terminal Language (`ui.js`)

What you will learn: Separation of presentation from logic. Every visual helper returns a string; the caller decides whether to print it. `chalk` is the only dependency.

Files to create: - `src/ui.js` - `test/ui.test.js`

Key concepts: - Every helper returns a string. The caller (usually `console.log`) prints it. - `ui.result(text)` classifies failure when the first line starts with `ERROR`, `DENIED`, or `FORBIDDEN`, or contains a non-zero `exit code N`. - `explainExit(code)` maps exit codes to plain-English meanings (1, 2, 3, 126, 127, 128–164, 9009, etc.). - `ui.forbidden(action, requested, reason)` must be conspicuous and greppable: `⛔ FORBIDDEN SAVE ../escape.txt - path escapes the workspace: ../escape.txt`. - Model output containing backslashes or colour-looking tags must print verbatim.

API TO BUILD:

```
ui.title(text)                // bold cyan
ui.field(label)               // green label, white ›
ui.kv(label, value)           // blue padded label + white value
ui.step(step, max, width=40)  // blue "── step X/Y ────"
ui.think(text)                // "think:" cyan + text
ui.tool(name, args)           // "tool →" white, name yellow, args white
ui.result(text)               // green/red result line, with details below
ui.forbidden(action, requested, reason)
ui.pass(text) / ui.fail(text)
ui.rule(label, width=44)      // bold magenta divider
ui.hookTitle(label)
ui.hookLine(icon, name, colour, when, live)
ui.hookField(label, value, colour)
ui.hookLive()
ui.hookNote(name, text)
ui.hookTrace(icon, name, colour, explanation, detail)
ui.hookFlow(steps)
ui.memory(text)
ui.exitNote(code, meaning)
ui.answerRule()               // bold green "── final answer ────"
ui.answer(text)
ui.warn(text) / ui.waiting(text) / ui.error(text) / ui.note(text)
ui.prompt(label) / ui.dim(text)
```

Acceptance criteria: - Every helper returns a string. - Styles become ANSI when TTY; strip cleanly or read back as original text when not. - Failed results are red; clean results are green. - Exit code 1 gets a description via `explainExit`. - Successful command shows first stdout line in a dim annotation. - `forbidden` produces bold red capitals.

Stage 2: Config & Model Client (`config.js`, `llm.js`)

What you will learn: `.env` discovery and precedence, fingerprinting, model normalisation, plain `fetch` to OpenRouter, `AbortSignal` composition.

Files to create: - `src/config.js` - `src/llm.js` - `test/config.test.js` - `test/llm.test.js`

Key concepts (config.js): - `PROJECT_ROOT = path.resolve(import.meta.dirname, '..')`. - `findEnvFile(startDir)`: nearest `.env` wins, walks up to filesystem root. In practice this means the script starts looking for `.env` in the repo root (where `package.json` lives) so that a `.env` placed there is found even when execution begins from deeper paths. - `loadDotEnv(file)`: supports `KEY=VALUE`, optional `export` prefix, `#` comments, quoted values, blank values ignored. - **Precedence:** `.env` overrides inherited shell variables (so a stale key in `~/.bashrc` does not outrank the project's key). - `fingerprint(secret)`: unset → `not set`; length ≤ 12 → all `*`; otherwise `<first5>...<last4>`. - `normalizeModel`: OpenRouter requires `vendor/model`. Map bare-name prefixes: `deepseek` → `deepseek`, `claude` → `anthropic`, `gpt` / `o1` / `o3` / `o4` / `chatgpt` → `openai`, `gemini` → `google`, `grok` → `x-ai`, `llama` → `meta-llama`, `qwen` → `qwen`, `mistral` / `mixtral` → `mistralai`. - `loadConfig(overrides, { envFile, startDir })`: returns full config with `apiKeySource`, `modelSource`, etc. - Defaults: model `deepseek/deepseek-v4.1-flash-1017`, `CMD_POLICY=ask`, `MAX_STEPS=12`, `REQUEST_TIMEOUT_MS=90000`, `WORKSPACE_DIR=workspace`, `MEMORY_FILE=src/memory.md`, `MEMORY_MAX_ENTRIES=60`, `MEMORY_PROMPT_CHARS=1200`.

Key concepts (llm.js): - `chat({ apiKey, model, messages, tools, signal, timeoutMs = 90000 })`. - Missing key → throw `OPENROUTER_API_KEY is not set...`. - Combine caller `signal` with `AbortSignal.timeout(timeoutMs)` via `AbortSignal.any`. - POST to `https://openrouter.ai/api/v1/chat/completions`. - Headers: `Authorization`, `Content-Type`, `X-Title: min-coding-js`. - Body: `{ model, messages, tools, max_tokens: 2048 }`. - On fetch timeout: throw `request timed out after <seconds>s`. - Non-2xx: hint 401 = bad key, 402 = out of credits. - Non-JSON body or `.error` field handled with truncated preview. - `sanitizeAssistant(message)`: rebuild with only `role`, `content`, `tool_calls`. Delete `content` when null and tool calls exist. Drop `reasoning` / `reasoning_content`.

Acceptance criteria: - `loadConfig` resolves `.env`, shell, overrides correctly. - `.env` beats stale shell key; shell used for names `.env` does not define. - Missing `.env` is not an error. - Fingerprint never exposes the whole key. - Bare model names normalised to `vendor/model`. - Override reported only when value genuinely differed. - `sanitizeAssistant` drops reasoning and rebuilds clean message. - LLM tests mock `fetch` to test timeout, error handling, and JSON parsing without a network.

Stage 3: The Simplest Chat Loop

What you will learn: wiring config, LLM, and UI into a runnable program. Reading user input. The first time you see the model reply.

Files to create: - `src/cli.js` (minimal version) - `test/input.test.js`

Files to modify: none yet (this is the first `cli.js`).

Key concepts: - This stage intentionally avoids tools, memory, hooks, and the full agent loop. The goal is **immediate gratification**. - `askTask({ input, output })`: readline loop. Prompt: `<green task> <white ›>`. Trim. Blank lines re-ask. Ctrl-C returns `null`. Use `for await (const raw of rl)` to support multi-line chunks. - Parse flags: `--help`, `--model`, `--max-steps`, and all remaining non-flags become the task string. - Load config; build `messages = [{ role: 'system', content: 'You are a helpful assistant.' }, { role: 'user', content: task }]`. - Call `chat()` from `llm.js`. - Print `ui.answer(message.content)`. - If no task argument and stdin is not a TTY, print a hint and exit 2. - Exit code 1 if the request fails (network/key error).

Acceptance criteria: - `node src/cli.js --help` prints concise usage with no API key needed. - `node src/cli.js "Say hello"` sends the task to OpenRouter and prints the model's reply. - No task + non-TTY stdin exits 2 with a helpful message. - Invalid/missing key exits 1 with a clear message. - `--model` flag overrides `.env`.

Stage 4: Conversation History

What you will learn: session state. The assistant remembers what you just said.

Files to create: none new.

Files to modify: `src/cli.js`

Key concepts: - Keep a `messages` array. Append user messages and assistant replies. - After a one-shot task, print a note that you can press Up or run again—but actually, implement an **interactive session loop** now. - Print `interactive session - type a task below (/quit exits)`. - Commands: `/quit`, `/exit`, `/new`. - `/new` clears `messages` and starts fresh (system prompt still re-added). - Trim blank input. - Every turn appends `user` → model call → append `assistant`. - Use the real model call, no tools yet.

Acceptance criteria: - Interactive session: follow-up questions reference previous context. - `/new` clears history. - `/quit` and `/exit` end the session. - Session prints `session over - N turn(s)` on exit. - One-shot still works.

Stage 5: System Prompt (`prompt.js`)

What you will learn: separating behavioural rules from control flow. Dynamic prompt composition.

Files to create: - `src/prompt.js` - `test/prompt.test.js`

Files to modify: `src/cli.js` (replace hard-coded system prompt with `buildSystemPrompt`)

Key concepts: - `systemRules(tools = TOOL_NAMES)` produces the prose behavioural contract. - Rules cover: look before you write, prefer `create_file` for new files, `edit_file` for small changes, `write_file` for whole-file replacement, old_string must be copied exactly, run what you wrote, do not invent tool results, be concise, one tool call per step, remember useful facts. - `buildSystemPrompt({ memory, log })`: rules + optional memory digest. - At this stage memory is not yet implemented, so the digest is always empty. - No memory → rules only.

Acceptance criteria: - Rules are injected as the first `system` message on every turn. - `/new` keeps the system prompt. - No tool names listed yet (since tools aren't built); the signature accepts `tools` but `cli.js` passes `[]` for now.

Phase B: Give the Chatbot Hands (Tools)

Now the model can touch the filesystem and run commands. We add tools one capability at a time.

Stage 6: Path Policy (`workspace.js`)

What you will learn: The foundation of agent safety: **reads are open; writes are fenced**. Path resolution, containment, symlink hardening.

Files to create: - `src/workspace.js` - `test/workspace.test.js`

Key concepts: - `PathError` exported. - `new Workspace(root)`: resolves root, `mkdir -p`, stores `fs.realpathSync(root)`. - `workspace.resolve(requested)`: strict write policy. Rejects: - empty/non-string; - absolute paths; - drive-relative (`C:foo`); - UNC (`\\server\share`, `//server/share`); - `~`, `$`, `%` prefixes; - forbidden first segments (`users`, `windows`, `program files`, `desktop`, `documents`, `downloads`, `appdata`, `onedrive`, `library`, `system32`, etc.). - Expands `~`, `%VAR%`, `${VAR}`, `$VAR` **before** containment checks. - Symlink safety: resolve the deepest existing ancestor with `fs.realpathSync`, append the remainder, then re-check containment. - `workspace.resolveRead(requested)`: permissive. Allows absolute paths, outside paths, `~`. - `rel` is `/`-separated; root is `.`.

Acceptance criteria: - Resolves legitimate relative paths; `rel` correct. - Refuses every escape shape. - Allows reads outside the workspace. - Symlinked folder inside workspace cannot smuggle a write outside.

Stage 7: Read-Only File Tools

What you will learn: The first tools the model can invoke. Argument validation. String observations (never thrown errors).

Files to create: - `src/tools.js` (partial: `list_files`, `read_file` only) - `test/tools.test.js` (read-only tests)

Files to modify: `src/prompt.js` (add `list_files` and `read_file` to the rules)

Key concepts: - OpenAI function-calling schema shape: `{ type: 'function', function: { name, description, parameters: { type: 'object', properties: {...}, additionalProperties: false } } }`. - `validate(name, rawArgs)`: raw may be JSON string or object; parse, type-check, integer-check. - `runTool(name, rawArgs, ctx)`: dispatch. Unknown tool → error string. - Handler exceptions become text observations (not thrown out of the loop). - `list_files`: - Params: `path` (default `'.'`), `depth` (default 3, max 6). - `resolveRead`. - Missing folder → `ERROR: no such folder: <rel>`. - Walk recursively, sorted, skip `.git`, `node_modules`, `.venv`, `__pycache__`. - Directories print `<rel>/`; files print `<rel> (<size>b)`. - Stop at `depth` and 200 lines. - End with `\n\n<dirs> folder(s), <files> file(s) under <start.rel>`. - `read_file`: - Params: `path` (required), `offset` (default 1), `limit` (default 200, max 2000). - `resolveRead`. - Missing → `ERROR: no such file: <rel>`; directory → `ERROR: <rel> is a folder - use list_files`. - Header: `"<rel> (<total lines>)"`. - Numbered lines right-aligned to width 4 followed by `|`. - Append `\n... [<n> more lines]` when truncated. - `offset` clamps ≥ 1; `limit` clamps 1..2000.

CLI changes at this stage: - `cli.js` now constructs `TOOL_SCHEMAS` for the two tools and passes them to `chat()`. - After the model reply, if `message.tool_calls` exists, execute the tools, append `role: 'tool'` results, and send back to the model for a final answer. - This is a **manual tool loop inside cli.js** (not `agent.js` yet). It teaches the ReAct pattern before we formalise it. - Print `ui.tool(name, args)` and `ui.result(result)`.

Acceptance criteria: - Model can call `list_files` and see directory listings. - Model can call `read_file` and see numbered file contents. - Bad tool arguments return text errors, not crashes. - Reads outside workspace succeed (e.g. `read_file` of `../package.json`). - Missing file returns `ERROR: no such file`. - Directory read returns `ERROR: is a folder`.

Stage 8: Sandbox + Write Tools

What you will learn: Defense in depth. The **only** place that prints `FORBIDDEN`. Honest limitations.

Files to create: - `src/sandbox.js` - `test/sandbox.test.js`

Files to modify: `src/tools.js` (add `create_file`, `write_file`, `edit_file`, `delete_file`); `src/cli.js` (wire sandbox)

Key concepts (sandbox.js): - `SANDBOX_DIR = path.resolve(import.meta.dirname, '..', 'workspace')`. - `ACTIONS = { SAVE: 'save', EDIT: 'edit', DELETE: 'delete' }`. - `ForbiddenError`: carries `action`, `requested`, `reason`; `name = 'ForbiddenError'`, `code = 'FORBIDDEN'`. - `new Sandbox({ workspace, root, log })`. - `sandbox.check(action, requested)`: delegates to `workspace.resolve()`, converts `PathError` to `ForbiddenError`, increments `refusals`, prints one `ui.forbidden(...)` line, then throws. - `sandbox.save(requested, content)`: `check(SAVE)`, mkdir parent, write file, return `{ existed, bytes }`. - `sandbox.remove(requested)`: `check(DELETE)`, `rmdirSync` for empty dirs, `unlinkSync` for files. Missing returns `{ missing: true }`. Deleting the sandbox root itself is refused. - `ForbiddenError` message format: `SAVE ../escape.txt - path escapes the workspace: ../escape.txt`. - `sandbox.summary()`: `"save, edit, delete - only inside <root>"`.

Key concepts (write tools): - `create_file`: `sandbox.check(SAVE)`, if exists → `ERROR: already exists - use edit_file or write_file`. Else write, return `created <rel> (<bytes> bytes)`. - `write_file`: `sandbox.check(SAVE)`, create parents, overwrite, return `created` or `overwrote`. - `edit_file`: - Params: `path`, `old_string`, `new_string`, `replace_all` (default false). - `sandbox.check(EDIT)`. Missing file → `ERROR`. - Empty `old_string` → `ERROR`. - Count exact occurrences using `indexOf` loop. - 0 occurrences → `ERROR: old_string not found... Read the file and copy exactly, including whitespace.` - >1 and no `replace_all` → `ERROR: old_string appears N times...` - Replacement literal: `source.split(oldStr).join(newStr)` or `replace` with a replacer function so `$1`, `$&` are **not** interpreted. - Return `edited <rel>: <n> replacement(s), first at line <line>`.

CLI changes: - `cli.js` creates one `Sandbox` per run and passes it to the tool handlers via `ctx`. - A write outside `workspace/` prints one ⛔ `FORBIDDEN` line and returns `FORBIDDEN: ...` to the model.

Acceptance criteria: - `create_file` writes inside workspace. - `create_file` refuses to clobber. - `write_file` overwrites. - `edit_file` works with exact string matching. - `edit_file` treats `$1` literally. - `edit_file` fails on missing file, empty old_string, ambiguous snippet. - Every escape shape refused for all

write/delete actions. - Deleting sandbox root refused. - Unknown action to `check()` throws `TypeError` (not a refusal). - A refused write prints exactly one `FORBIDDEN` line.

Stage 9: Command Execution (`run_command`)

What you will learn: Running shell commands safely. Hard deny-lists, environment allow-lists, policy layers (`deny` / `ask` / `allow`).

Files to create: - `test/shell-sandbox.test.js`

Files to modify: `src/sandbox.js` (add `checkCommand`, `tokenizeCommand`, `shellWriteTargets`); `src/tools.js` (add `run_command`)

Key concepts (shell target scanning): - `tokenizeCommand(text)`: split on whitespace honouring single/double quotes. Emit operators as tokens: `>`, `>>`, `<`, `|`, `||`, `&`, `&&`, `;`. Quotes stripped. - `shellWriteTargets(command)`: - Redirections `>` / `>>`: following token is a write target. Handle `2> file`. - `DEST_LAST` commands (`copy`, `cp`, `move`, `mv`, `xcopy`, `ren`, `rename`, `ln`, `install`, `scp`, `rsync`): last positional is destination (require ≥2 positionals). - `robocopy`: positional arg 2 is destination. - `ALL_POSITIONAL` commands (`del`, `erase`, `rd`, `rmdir`, `rm`, `md`, `mkdir`, `touch`, `tee`, `unlink`, `truncate`, `shred`): every positional is a target. - Downloaders: `curl` `-o` / `--output` / `-O` / `--remote-name`; `wget` `-O` / `--output-document`. - PowerShell: `out-file`, `set-content`, `add-content`, `new-item`, `ni`, `remove-item`, `copy-item`, `move-item`, `rename-item`; target flags `-destination`, `-dest`, `-path`, `-filepath`, `-literalpath`. - Strip `.exe`, `.cmd`, `.bat`, `.com`, `.ps1` from command names; compare case-insensitively. - A switch is `--flag`, `-f`, or Windows `/x`; a POSIX path like `/home` is not a switch.

Key concepts (`run_command`): - Trim; empty → `ERROR: command is required`. - Hard deny-list **before** policy and approval: - recursive delete of root path, `del /...s` or `/q`, `format`, `diskpart`, `shutdown`, `reboot`, `git push --force` / `-f`, `curl` / `wget` piped into `sh` / `bash`, fork bomb `:(){ ...};:`, `> /dev/sd...`. - Call `sandbox.checkCommand(command)` before asking human for approval. - Policy: `deny` → `DENIED`; `allow` → proceed; `ask` → confirm via readline (only `y` / `yes` = allow; no TTY = deny). - Execute with `node:child_process.exec` in `workspace.root`, `timeout: 30000`, `maxBuffer: 1 MB`, `windowsHide: true`, `encoding: 'utf8'`. - Environment: `safeEnv(source)` keeps only `PATH, PATHEXT, HOME, USERPROFILE, TMPDIR, TEMP, TMP, SYSTEMROOT, COMSPEC, SHELL, LANG, LC_ALL, TERM`. Child **must not** inherit harness env (no `OPENROUTER_API_KEY` leak). - Result format: `exit code <n>`, optional `stdout:`, optional `stderr:`. Add `(no output)` if both empty. - Clip each stream to 6,000 chars; append `... [<n> chars truncated]`. - Timeout appends `[tool] command timed out after 30s` to stderr and returns non-zero code.

CLI changes: - Add `--yes` flag to auto-approve commands (sets policy to `allow` for that run). - Add `--allow-commands` alias. - `makeConfirm()` function in `cli.js`.

Acceptance criteria: - `run_command` returns exit code, stdout, stderr. - Hard-denied commands refused even with `--yes`. - `CMD_POLICY=deny` refuses all commands. - Shell copy to Desktop refused before running. - Redirect outside refused before running. - Quoted arguments with spaces stay one token. - Child env does not contain `OPENROUTER_API_KEY`. - Timeout handled gracefully. - `safeEnv` keeps only allow-listed names.

Phase C: Memory & Observability

Now the agent can remember things across runs, and we can observe its lifecycle.

Stage 10: Long-term Memory (`memory.js`, `remember`, `recall`)

What you will learn: File-as-database. No cache, no vector store. A student can open `src/memory.md` and see exactly what the agent knows.

Files to create: - `src/memory.js` - `src/memory.md` (initial store) - `test/memory.test.js`

Files to modify: `src/prompt.js` (inject memory digest); `src/tools.js` (add `remember`, `recall`); `src/cli.js` (wire memory)

Key concepts: - Format: `- [tag] text _(YYYY-MM-DD)_`; tag ∈ {`fact`, `pref`, `fix`, `gotcha`}. - `parseMemory`: accepts `-` or `*`, optional date, normalises whitespace. Ignore prose/headings. - `Memory({ root, file, maxEntries=60, maxEntryChars=300, rememberChars=2000, promptChars=1200, promptEntries=12 })`. - `remember(text, { tag })`: normalise, truncate to `rememberChars`. Empty → error. Duplicate refreshes date instead of appending. New appends; drops oldest if over `maxEntries`. - `forget(match)`: delete entries whose text contains match (case-insensitive). - `search(query, { limit=20 })`: AND over words; every word must match text or tag. Empty query = all. Newest first. - `digest({ entries, chars })`: eager path injected into system prompt. Newest `promptEntries` first. Add lines while they fit `promptChars`. If hidden: `(<n> older memory/memories not shown - search <label> with the recall tool)`. - `summary()`: `<label> (<n> memory/memories[, newest YYYY-MM-DD])`.

Tool contracts: - `remember`: params `text` (required), `tag` (optional). - No memory store → `ERROR: no memory store in this run (started with --no-memory)`. - Empty → `ERROR: nothing to remember - the note was empty`. - Duplicate → `already remembered in <label> (<total> total), date refreshed: <formatted entry>`. - New → `remembered in <label> (<total> total[, dropped the N oldest]): <formatted entry>`. - `recall`: params `query` (optional), `limit` (default 20). - No memory → same error. - No hits → `no memories in <label> match "<query>" (<size> stored)`. - Hits → formatted entries.

CLI changes: - Add `--memory` side-door: prints the store, digest, and raw file; exits 0. - Add `--no-memory` flag. - Memory file location configurable via `MEMORY_FILE`.

INITIAL `SRC/MEMORY.MD`:

```
# memory

Long-term notes this agent keeps between runs. One bullet per memory:
`- [tag] text _(YYYY-MM-DD)_` where tag is fact | pref | fix | gotcha.

This file IS the store - edit it by hand, delete a wrong line, or commit it.
The agent reads a bounded slice at the start of a run and can search the rest
with the `recall` tool.

- [pref] The user's name is Craig. _(...)_
- [pref] The user lives in Brighton, UK. _(...)_
- [gotcha] The run_command shell in this sandbox is Windows cmd.exe, so Unix tools like `cat`, `ls` may fail; use
`type`, `dir` etc. instead. _(...)_
- [pref] The user is a Python developer. _(...)_
- [pref] The user has two dogs. _(...)_
```

Acceptance criteria: - Write and read back survive a new `Memory` instance. - Duplicate refreshes date; no duplicate bullets. - `maxEntries` drops oldest. - Long memory truncated to per-entry cap. - Empty/whitespace note refused. - Digest bounded, newest first, states hidden count. - Empty/disabled store adds and logs nothing. - `forget` removes matches. - Hand edits picked up immediately. - `remember` / `recall` work through tools. - No memory store returns error string, not a throw. - `--memory` prints digest and exits 0. - `--no-memory` disables memory for that run.

Stage 11: Lifecycle Hooks (`hooks.js`)

What you will learn: Extension points that are **asked**, not merely notified. Observe, rewrite, block, replace. Pure payload functions—no model calls, no tools.

Files to create: - `src/hooks.js` - `test/hooks.test.js`

Files to modify: `src/cli.js` (add hook wiring to the manual tool loop)

Key concepts: - Nine events, exact names, icons, colours:

#	NAME	ICON	COLOUR	WHEN	CAN
1	`SessionStart`	🚀	cyan	once, before first model call	refuse, note
2	`UserPromptSubmit`	✍️	blue	once, as task enters loop	rewrite task, block
3	`PreModel`	🧠	magenta	before every model call	observe
4	`PostModel`	💬	cyan	after every reply	observe
5	`PreToolUse`	🛠️	yellowBright	before every tool call	**block**
6	`PostToolUse`	📦	green	after every tool call	**replace observation**
7	`Stop`	🏁	greenBright	model answered, loop ending	observe
8	`StepLimit`	⏳	yellow	maxSteps reached	observe
9	`SessionEnd`	🔒	red	always, finished or not	observe

- Four deciding hooks can change the run: `SessionStart`, `UserPromptSubmit`, `PreToolUse`, `PostToolUse`.
- Return shapes:

- `{ note: 'text' }` → print `ui.hookNote`.
- `{ task: 'new text' }` → rewrite task (UserPromptSubmit only).
- `{ replace: 'text' }` → replace observation (PostToolUse only).
- `{ block: true, reason }` → block. First block wins.
- Thrown hooks reported and ignored; never fatal.
- `Hooks` API: `on(name, fn, { id, once })`, `off(name, id)`, `count(name)`, `has(name)`, `attached()`, `run(name, payload)`.
- `traceHooks({ log, detail = hookDetail, explain = true })`: returns a `Hooks` with one tracer per event. Prints `<icon> [HOOK:<Name>] <when> - <detail>` via `ui.hookTrace`.
- `printHookCatalogue({ log, hooks })`: prints full table.
- `HOOK_SUMMARY = { total: 9, deciding: 4, names: HOOK_NAMES }`.

CLI changes: - Add `--hooks` side-door: prints catalogue and exits 0. - Add `--trace-hooks` (default on) and `--no-hooks`. - In the manual tool loop inside `cli.js`, fire hooks at the key moments: - `SessionStart` before first model call. - `UserPromptSubmit` before the loop. - `PreModel` before each model call. - `PostModel` after each reply. - `PreToolUse` before each tool call. - `PostToolUse` after each tool call. - `Stop` when model answers without tools. - `StepLimit` when max steps hit. - `SessionEnd` on every exit.

Acceptance criteria: - Every hook has icon, colour, when, gets, may, why. - Catalogue prints all 9; marks attached hooks. - Unknown hook name throws. - Handlers run in order; first block wins. - Throwing hook reported, not fatal. - Notes print; `once` handlers detach. - `off` by id works. - Trace prints `[HOOK:Name]` by default. - `--no-hooks` silences trace. - `--hooks` prints catalogue with no key.

Phase D: The Agent Loop & Polish

Now we extract the loop into a proper library, add the offline demo, and polish the CLI with sessions.

Stage 12: The ReAct Loop (`agent.js`, `heartbeat.js`)

What you will learn: Formalising the conversation loop as a testable library. The seven beats per step.

Files to create: - `src/agent.js` - `src/heartbeat.js` - `test/agent.test.js`

Files to modify: `src/cli.js` (replace manual tool loop with `runTask` calls)

Key concepts: - `runTask({ task, workspace, sandbox = null, commandPolicy, maxSteps, chatFn, confirm, hooks = null, memory = null, history = [], log = console.log, heartbeatMs = 5000 })`. - Return: `{ ok, answer, messages, steps }` or `{ ok: false, reason, messages }`. - System prompt rebuilt every task (never carried in history). Memory written by previous task appears in next task's fresh system prompt. - Tool context (`ctx`): `{ workspace, sandbox, commandPolicy, memory, confirm }`. - `confirm` defaults to `async () => 'deny'`. - `hooks ?? new Hooks()` so every call site can call `hook.run(...)` unconditionally. - `agent.js` executed directly prints: `agent.js is the loop, not the program. Run the CLI instead: node src/cli.js --help` - Do not import `cli.js` from `agent.js`.

The seven beats per step (for step = 1..maxSteps):

1. Print blank line + `ui.step(step, maxSteps)`.
2. Fire `PreModel` with `{ step, maxSteps, messages, tools: TOOL_SCHEMAS }`.
3. Start `heartbeat(log, heartbeatMs)` and call `chatFn({ messages, tools: TOOL_SCHEMAS })`. - Throw: stop heartbeat, print `ui.error`, fire `SessionEnd`, return `{ ok: false, reason: 'request_failed', messages }`. - Success: stop heartbeat.
4. Extract `choice = reply?.choices?.[0] ?? {}`, `message = choice.message ?? {}`. Fire `PostModel` with `{ step, message, finishReason: choice.finish_reason }`.
5. **Truncation guard:** if `finish_reason === 'length'` and no tool calls: - append assistant message; - append user: `"You were cut off by the length limit. Continue with smaller tool calls."`; - print `ui.warn(...)`; - `continue` to next step. Never dispatch truncated tool calls.
6. Append `sanitizeAssistant(message)` to messages. If `content` non-empty, print `ui.think(...)`.
7. `calls = message.tool_calls ?? []`. - **No calls:** print `ui.answerRule()` + `ui.answer(...)`. Fire `Stop`, then `SessionEnd`. Return `{ ok: true, answer, messages, steps: step }`. - **Calls:** dispatch each in order:
   1. `name = call.function?.name ?? '(unnamed)'`; `rawArgs = call.function?.arguments ?? '{}'`.
   2. Print `ui.tool(name, rawArgs)`.
   3. Fire `PreToolUse` with `{ name, args: rawArgs, step }`. If blocked, observation = `DENIED: a PreToolUse hook blocked this call (<reason>). Try another approach.` Otherwise `await runTool(name, rawArgs, ctx)`.
   4. Print `ui.result(result)`.
   5. Fire `PostToolUse` with `{ name, result, step, ok: !/^(ERROR|DENIED)\b/.test(result) }`. If `post.replace !== null`, use replacement as observation.

6. Append exactly one `{ role: 'tool', tool_call_id: call.id, content: String(post.replace ?? result) }`.

After loop exhausts maxSteps: - Print `ui.error('stopped: hit the <maxSteps>-step limit')`. - Fire `StepLimit` then `SessionEnd`. - Return `{ ok: false, reason: 'max_steps', messages }`.

Entry guards: - `SessionStart` blocked → print error, fire `SessionEnd`, return `{ ok: false, reason: 'blocked_by_hook', messages }`. - `UserPromptSubmit` blocked → same. - `UserPromptSubmit` rewrite → print dim note, replace `messages[taskIndex]`.

Heartbeat: - `heartbeat(log, everyMs = 5000)` returns `{ stop }`. - Every interval prints `ui.waiting(' … still waiting for the model (<seconds>s)')`. - Call `timer.unref?.()` so it never holds the process open.

Acceptance criteria: - Loop runs tools then stops on plain answer. - Failing tool result fed back so model can correct. - Loop capped at `maxSteps`. - Truncated reply not dispatched; model asked to continue smaller. - Heartbeat prints waiting lines during slow calls. - Blocked hooks produce correct exit paths. - Step limit fires `StepLimit` then `SessionEnd`. - Session tests: follow-up task sees previous conversation but fresh system prompt. - Direct execution prints hint, does not import CLI.

Stage 13: Offline Demo (`demo.js`)

What you will learn: Fake model injection. Scripted runs prove every guard holds with no API key.

Files to create: - `src/demo.js`

Files to modify: `src/cli.js` (add `--demo` flag); `test/agent.test.js` (add demo assertions)

Key concepts: - `demoChat()`: returns an async `chatFn` that walks `DEMO_SCRIPT`. - Tool call shape: `{ id, type: 'function', function: { name, arguments: JSON.stringify(args) } }`. - `DEMO_SCRIPT`: 14 steps + final answer: 1. `list_files { path: '.' }` 2. `create_file { path: 'demo/hello.js', content: "const agent = 'tiny agent';\nconsole.log(\` Hello from a ${agent}\`);\n" }` 3. run_command `4.` edit_file }', new_string: 'Hello from a ${agent}, edited by a tool call' `5.` run_command `6.` write_file `→ expect FORBIDDEN 7.` read_file `→ allowed 8.` write_file `→ expect FORBIDDEN 9.` run_command `→ expect FORBIDDEN 10.` run_command `→ expect DENIED 11.` run_command `→ expect absent 12.` run_command `→ expect exit code 1 13.` remember `14.` recall ` - Then final answer summarising everything.

- Each assistant step has a short natural-language `content` before its tool call.
- `maxSteps = DEMO_SCRIPT.length + 1` = 15.
- `sandboxScorecard(messages)`: maps `tool_call_id` → tool result, runs regex checks, returns `{ rows: [{ ok, label, seen }], passed, total }`.
- Scorecard checks:
- s6: `/^FORBIDDEN: SAVE \.\.\/escape\.txt/`
- s7: `/^\.\.\/package\.json \(\d+ lines\)/`
- s8: `/^FORBIDDEN: SAVE ~\/Desktop\/pwned\.txt/`
- s9: `/^FORBIDDEN: COMMAND COPY /`
- s10: `/^DENIED: blocked by policy/`

- s11: `/OPENROUTER_API_KEY=absent/`
- s12: `/^exit code 1\b/`

CLI changes: - `--demo` sets scripted chat, `maxSteps = 15`, auto-approves commands, and prints scorecard. - Demo run exits 0 on 7/7 checks; otherwise `process.exitCode = 1`.

Acceptance criteria: - `node src/cli.js --demo` completes 14 steps + final answer. - Creates/edits/runs `workspace/demo/hello.js`. - Refuses `../escape.txt`, `~/Desktop/pwned.txt`, shell copy to Desktop. - Allows read of `../package.json`. - Denies `git push --force`. - Shows `OPENROUTER_API_KEY=absent` from child. - Reports exit code 1 with explanation. - Remembers and recalls. - Scorecard: **7/7 checks held**. - Exits 0 on success.

Stage 14: Sessions, History Trimming & Full CLI Polish

What you will learn: session management, `trimHistory`, full banner with config fingerprinting, and exit codes.

Files to create: none new.

Files to modify: `src/cli.js` (major polish); `test/session.test.js`

Key concepts: - `MAX_HISTORY = 24`. - `trimHistory(messages, max = MAX_HISTORY)`: - Drop system messages. - Keep at most last `max` messages. - Move `start` forward until it lands on a `user` message. Providers reject history starting with `tool` or ending with unanswered assistant `tool_calls`. - If no safe boundary, return `[]`. - Interactive session (`runSession`): - Loop: `askTask` → command or task → `runTask({ ...base, task: line, history })` → `history = trimHistory(result.messages)`. - Every task gets a fresh step budget (`runTask` is called fresh each time). - Print summary after each task: divider, pass/fail, message count hint, retry hint. - On exit: `session over - N task(s) run.` - Full banner (before step 1): `min-coding-js workspace : <root> sandbox : save, edit, delete - only inside <root> (reads: anywhere) model : <model> (from <source>) | scripted (no network) key : <fingerprint> from <source> | - in demo env file : <path> | <path> (missing) commands : allow | ask | deny timeout : <seconds>s per model call tools : <comma-separated TOOL_NAMES> memory : <summary> | off (--no-memory) hooks : on: each of 9 prints as [HOOK:name] ... | 9 available, not printed (--no-hooks)` - Warnings: - live mode + model lacks `/` → `warning: model is not an OpenRouter slug...` - config normalised bare name → dim note with final slug. - Exit codes: - 0: finished / answer / session ended normally. - 1: model request failed or fatal error. - 2: no task and stdin not a terminal. - 3: step limit reached. - `taskExitCode(result)`: - `!ok && reason === 'request_failed'` → 1 - `!ok && reason === 'max_steps'` → 3 - otherwise 0 - `main()` catches fatal errors, prints `fatal: <stack>`, sets `process.exitCode`. - Only call `main()` when `cli.js` is the entry file (`import.meta.url === pathToFileURL(process.argv[1]).href`).

Acceptance criteria: - `/new` clears conversation history but leaves workspace and memory untouched. - `/quit` and `/exit` end session. - Follow-up task sees previous conversation but fresh system prompt containing latest memory. - Every task starts its own step budget. - A hook rewriting the task rewrites the new task, not history. - No task + non-TTY stdin exits 2. - Step limit reached exits 3. - Live mode without key exits 1. - Banner prints on every run. - Exit note prints plain-English meaning for non-zero codes.

Phase E: Hardening & Docs

Stage 15: Full Test Suite

What you will learn: offline testability. Fake models, temporary directories, injected loggers.

Files to create: any remaining test files, plus ensure all prior test files are comprehensive.

Files to modify: ensure all `test/*.test.js` files are complete per the original specification.

Required suites (refer to original spec for full detail): - `test/workspace.test.js` — path policy, containment, symlink safety. - `test/sandbox.test.js` — refusals, root deletion, tool-layer conversion. - `test/tools.test.js` — all eight tools, validation, safeEnv, policy. - `test/shell-sandbox.test.js` — tokenization, write-target scanning. - `test/agent.test.js` — loop behaviour, truncation, heartbeat, demo guards. - `test/hooks.test.js` — registry, blocking, replacing, tracing, catalogue. - `test/memory.test.js` — round-trip, duplicates, bounds, digest, forget, search. - `test/prompt.test.js` — rules, injection, logging. - `test/llm.test.js` — timeout, errors, sanitization. - `test/config.test.js` — .env precedence, fingerprint, model normalisation. - `test/session.test.js` — commands, trimming, follow-up tasks. - `test/ui.test.js` — styling, verbatim output, result classification. - `test/input.test.js` — askTask, blank lines, Ctrl-C.

Acceptance criteria: - `npm test` runs fully offline. - All suites pass. - No real network calls. - No API key required.

Stage 16: Documentation & Final Acceptance

What you will learn: writing a README that teaches architecture, not just commands. Final integration.

Files to create: - `README.md` - `demo_prompt.md` - `.env.example` (if not already present)

README content (teachable structure): - One-paragraph summary. - Quick start: `npm install`, copy `.env.example` to `.env`, set key, `npm start`, `npm run demo`. - The loop diagram and the seven beats. - File-by-file tour, with `agent.js` first. - The eight tools and what each is for. - The read/write asymmetry and the shell limitation. - The nine lifecycle hooks, with a table. - Long-term memory: file format, eager digest vs lazy recall, `--memory`. - Sessions and history trimming. - Configuration table with defaults and precedence. - Exit codes. - Safety model: workspace fence, hard denies, environment allow-list, command policy, and explicit "this is not a container" statement. - Examples: a Desktop write that ends in `FORBIDDEN`, a prompt that ends with `remember`.

`demo_prompt.md`**:** - Copy-pasteable prompts including: - `create hello.js that prints today's date and time, run it and save...` - `save a file test.txt that contains 'HELLO WORLD' and save to the desktop` (must end in FORBIDDEN). - `npm run demo`, `node src/cli.js --memory`, `node src/cli.js --demo`. - `work out how this package is tested, run the tests, and remember the command...`

Final acceptance criteria (from spec):

1. `npm install` succeeds with only `chalk` as a runtime dependency.
2. `npm test` runs fully offline and all suites pass.

3. `node src/cli.js --help` prints usage without needing a key.
4. `node src/cli.js --hooks` prints the full nine-hook catalogue without needing a key.
5. `node src/cli.js --memory` prints the store and the exact digest that would be injected.
6. `node src/cli.js --demo` completes the 14-step script with **7/7 checks held**.
7. Live mode with a valid key runs a task through OpenRouter.
8. Live mode without a key prints missing-key error and exits 1.
9. No task + no TTY exits 2.
10. Task that never yields answer stops at step limit and exits 3.
11. Write outside `workspace/` prints one ⛔ `FORBIDDEN` line and returns `FORBIDDEN:` observation without touching disk.
12. Read outside `workspace/` succeeds.
13. `CMD_POLICY=deny` refuses all commands; hard-denied commands refused even with `--yes`.
14. Child shell cannot read `OPENROUTER_API_KEY`.
15. Memory survives process restarts; duplicate refreshes; file bounded; hand edits picked up.
16. Every normal run prints `[HOOK:Name]` trace by default; `--no-hooks` silences it.
17. `/new` clears conversation history but leaves workspace and memory untouched; `/quit` and `/exit` end session.
18. Follow-up task sees previous conversation but freshly rebuilt system prompt containing latest memory.
19. `agent.js` executed directly prints hint and does not run CLI.
20. No framework, no agent SDK, no database, no hidden global state.

Checklist

STAGE	MODULE	WHAT YOU CAN DO AFTER THIS STAGE	ACCEPTED
0	Scaffold	`npm install` works	☐
1	`ui.js`	Print styled terminal output	☐
2	`config.js` + `llm.js`	Load `.env`, call OpenRouter	☐
3	Minimal `cli.js`	**Chat with the model** 🎉	☐
4	History in `cli.js`	Have a **multi-turn conversation**	☐
5	`prompt.js`	Inject behavioural rules	☐
6	`workspace.js`	Fence writes, open reads	☐
7	Read tools	Model **lists and reads files**	☐
8	Sandbox + write tools	Model **creates and edits files** safely	☐
9	`run_command`	Model **runs shell commands** with policy	☐
10	`memory.js`	Model **remembers and recalls** across runs	☐
11	`hooks.js`	Observe/rewrite/block lifecycle events	☐
12	`agent.js`	Extract the loop into a **testable library**	☐
13	`demo.js`	**Offline demo** proves all guards hold	☐
14	Full CLI polish	**Sessions**, history trimming, banner, exit codes	☐
15	Tests	`npm test` passes fully offline	☐
16	Docs	README teaches the architecture	☐